Sanskrit-to-English Neural Machine Translation

Natural Language Understanding — Assignment 2

Sanidhya Mittal

Roll No: G25AIT1151

System: IndicTrans2 (distilled 200M) Transformer encoder–decoder, fine-tuned with LoRA

Code Available on : https://github.com/Sanidhya1993/G25AIT1151-NLU-NMT-Assignment2

Headline results (public split)

Split	BLEU (NLTK)	BERTScore F1	Inference time	Total parameters
Dev (1000)	0.2611	0.6213	128.7 s	211,780,608
Test (1000)	0.2500	0.6216	140.8 s	211,780,608

The default BLEU corpus_bleu (uniform 4-gram weights) is used. To calculate the BERTScore F1 score, the official bert-score library is used with **rescale_with_baseline=True**. Inference time is total wall-clock to translate a full 1000-sentence split on an **RTX 3060** Laptop GPU using KV cache. The model is fine-tuned using LoRA, which requires **12,976,128** trainable parameters, with 5.77% of the model parameters trainable. The model parameters are then merged with the adapters before inference, resulting in a total of **211,780,608** parameters in the deployed model.

1. Introduction

The present work explains an End-to-end Neural Machine Translation (NMT) system to translate Sanskrit source sentences to English. Sanskrit is a low-resource, morphologically rich language: morphology marks number, case, tense and mood, and Sanskrit freely compounds words, making translation to English more difficult than for high-resource pairs. The data set used in this assignment is also a very mixed one in domain, comprising both software/technical help texts, scriptural texts, and sentences in general, thus further challenging a model learned from a limited data set.

The system is based on a sequence-to-sequence model (a Transformer encoder–decoder with cross-attention and beam-search decoding). The weights for the encoder–decoder are initialised from the publicly disclosed pretrained model **ai4bharat/indictrans2-indic-en-dist-200M** and then fine-tuned for the task using **Low-Rank Adaptation (LoRA)**: small low-rank adapter matrices are inserted into the attention and feed-forward projections, and only the adapter matrices are trained. It is a parameter-efficient fine-tuning **(PEFT)** approach that fine-tunes less than 6% of the parameters. It's allowed to use pretrained models under the rules of the assignment; here's the only pretrained model utilized, and no external API for translation is called anywhere. The rest of the pipeline (data alignment, Indic and Unicode preprocessing, development-BLEU checkpoint selection, beam search decoding and metric computation) is straight out of the submitted notebook.

The trained system is tested using the three metrics defined in the assignment: NLTK BLEU, BERTScore F1, efficiency (inference time, number of parameters). It obtains a **BLEU score of 0.2500 and a BERTScore F1 of 0.6216** on the public test split and translates the entire 1000-sentence test set in 140.8 seconds. A full-fine-tuning variant was also performed and compared, with the LoRA configuration reported here outperforming on all quality metrics (Section 6).

2. Architecture

Architecture is a Transformer encoder–decoder with weights being initialized from the pretrained model **ai4bharat/indictrans2-indic-en-dist-200M**, which is the distilled 200M IndicTrans2 model for Indic to English translation. The model is ingested via the **Hugging Face AutoModelForSeq2SeqLM** interface with trust_remote_code=True to download the IndicTrans2 modelling code. The pretrained initialisation is necessary because of the small number of training pairs (~10k; assignment rules state this checkpoint must be published).

2.1 Encoder

The Encoder consists of a stack of Transformer layers, which converts the tokenised Sanskrit input into a series of hidden states in the context of the input. In each layer, they aggregate multi-head self-attention (where each source token can attend to every other source token) with a position-wise feed-forward sub-layer, feeding each token through a sub-layer with residual connections and layer normalisation around it. Positional information is added at embedding stage as the self-attention is order-invariant. The padding positions are not involved in calculating attention, but are included with the source token ids and attention mask that indicates the padding.

2.2 Decoder

The decoder is an auto-regressive Transformer stack to generate an English translation, token by token. The three components of each decoder layer are: masked self-attention: which only allows the decoder to pay attention to past tokens (the causal mask), cross-attention: which allows the decoder to attend to parts of the encoder, and a feed-forward sub-layer (with residual connections and layer normalisation). The sub-word vocabulary is used in a final linear projection followed by softmax, which gives the next-token distribution. In training the gold target sequence is shifted to the right and the decoder input contains the gold target sequence; the model is trained to predict the next token at each position with teacher forcing.

2.3 Attention

The "attention" mechanism is at the heart of all three of the above. Scaled dot-product attention calculates a compatibility score between the query and the keys, normalizes them with a softmax, and gives a weighted sum of values: each head works on a different sub-space of the model to attend to a different one. Encoder self-attention is used to build source context, decoder masked self-attention is used to build target context without leaking future tokens, and encoder–decoder cross-attention is used as a bridge to allow each generated English token to be influenced by the whole of the Sanskrit sentence. This “cross-attention” is what makes output words match the relevant input content.

2.4 LoRA adaptation

It only updates the linear layers with small matrices ***ΔW = BA***, where A, B are small matrices of rank r, while the other linear layers are frozen with the pretrained weights. The product BA is scaled by α/r, and added to the original weights, only A and B are trained, and at inference, only ***α/r*** is multiplied by BA. The rank (r) is 32, the ***α*** is 64, and the adapter dropout rate is 0.05, where LoRA is applied to the attention query, key, value and output projections ***(q_proj, k_proj, v_proj, out_proj)*** and to the feed-forward projections (fc1, fc2) in every layer of the encoder and decoder. This reduces the number of trainable parameters to 12,976,128 parameters, which is 5.77% of the total model.This leaves 12,976,128 parameters trainable, or 5.77% of the model, with the rest of the parameters, ~94%, being frozen. The adapters are trained and then merged into the base weights using merge_and_unload(), resulting in the model for evaluation and submission being a single standard 211,780-parameter Transformer with no runtime adapters overhead.

```
Injecting LoRA adapters (parameter-efficient fine-tuning)...
Model loaded: ai4bharat/indictrans2-indic-en-dist-200M
Fine-tuning mode: LoRA (PEFT)
Total parameters: 224,756,736
Trainable parameters: 12,976,128
Trainable fraction: 5.773%
```

Figure 1. Breakdown of the parameters at the moment of loading. LoRA introduces 12.98M trainable adapter parameters on top of the 211.78M frozen base (5.77% trainable); during training, the adapters are fused with the base, but before the inference, the adapters are fused away.

2.5 Beam-search decoding

The decoder performs beam search at inference, whereas it performs greedy decoding at training time. Beam search maintains the num_beams most likely partial hypotheses at each step, and expands them: this leads to less myopic error in greedy decoding and generally better adequacy and fluency. The configuration employed is a beam width of 5, length penalty of 1.0, a maximum of 128 newly generated tokens, and early stopping on the occurrence of an end-of-sequence token by all of the beams. To keep inference fast, key–value (KV) caching is enabled, reusing previously calculated attention states across decoding steps; if a runtime incompatibility is detected, it falls back to non-cached generation. The official IndicTrans2 recipe is followed here: inputs are prepared using the IndicProcessor in inference mode, and the output strings are finished with its postprocess_batch step, before scoring.

3. Training Procedure

3.1 Data and alignment

The Sanskrit and English CSV files are read using a UTF-8 (and UTF-8-BOM fallback) reader and tie on the common Source id. The training set has 9,982 aligned sentence pairs, and the development and test sets have 1,000 sentences each, respectively. A brief profile of the training data reveals that the Sanskrit sources' average number of whitespace tokens is 9.7 (max of 55) while the English targets' average number of tokens is 12.7 (max of 116) and the two distributions are very close, as illustrated in Figure 4.

3.2 Preprocessing and tokenisation

Repeated whitespace has been collapsed and all text is normalised (NFC). Inside the IndicTrans2 IndicProcessor, source and target strings are sent through the normaliser and the model-specific language tags san_Deva (source) and eng_Latn (target) are added to them as well. Training text runs the processor in the training mode, decoding uses a separate processor that is running in the inference mode plus post-processing. The text is tokenized up to 128 tokens - padded by the model's own sub-word tokeniser. For the label tensor, padded positions are replaced with -100 to indicate that they should be ignored in the loss.

3.3 Hyperparameters

Hyperparameter	Value	Hyperparameter	Value
Fine-tuning	LoRA (PEFT)	Train batch size	4
LoRA rank / α	32 / 64	Infer batch size	8
LoRA dropout	0.05	Beam width	5
LoRA targets	q,k,v,out_proj; fc1,fc2	Length penalty	1.0
Optimiser	AdamW	Max seq / new tokens	128
Learning rate	3×10^{-4}	Label smoothing	0.1
Weight decay	0.01	Grad clip norm	1.0
LR schedule	linear + 10% warmup	Mixed precision	FP16
Epochs (max)	6	Random seed	42
Early stopping	patience 2 (dev BLEU)	Selected checkpoint	epoch 6

3.4 Optimisation

AdamW is used for training, with disabled bias and layer-norm parameters, only the LoRA adapter parameters are passed to the optimiser while the base weights are frozen. The learning rate is scheduled on a linear schedule, with 10% warmup over the number of steps. LoRA has the advantage of being able to use a relatively higher learning rate than full fine tuning: $\mathbf{3 \times 10^{-4}}$ was used here, which is an order of magnitude higher than $\mathbf{3 \times 10^{-5}}$ for full fine tuning at this batch size. When using Mixed-precision training (FP16), gradients are clamped to the max norm of 1.0 and memory usage is reduced while the training is accelerated by each step on the RTX 3060 Laptop GPU.

3.5 Regularisation and model selection

The following three factors regularise the training: The first is that LoRA is already a very powerful regulariser: by constraining the updates to the low rank space, the model has limited capacity to memorise the **~10k** pairs, improving the generalisation performance. Second, label smoothing on 0.1 is used via a custom cross-entropy criterion (with an ignore index of -100), this helps to smooth the target distribution and to prevent over-confident predictions. Third, model selection is done on dev-set BLEU computed every epoch, not on the final epoch; best checkpoint saved and re-loaded for evaluation; an early stopping patience of 2 stops training if there is no improvement in dev-set BLEU. When performing this setup development BLEU gradually increased to achieve the highest value at **epoch 6 (0.2611)**.

4. Results

The following table shows the assignment metrics for the development and public test splits. The default NLTK corpus BLEU; F1 with baseline rescaling (BLEU); efficiency (total inference time per split of 1000 sentences; total number of parameters in the merged model).

Metric	Development (1000)	Test (1000)
BLEU (NLTK, no weights)	0.2611	0.2500
BERTScore F1 (rescaled)	0.6213	0.6216
Inference time (total)	128.7 s	140.8 s
Total parameters (merged)	211,780,608	211,780,608
Trainable parameters	12,976,128	12,976,128

Efficiency notes: *the merged model has 211,780,608 parameters; only 12,976,128 (5.77%) were trained. Inference of 140.8 s on 1000 sentences is about 0.14 s per sentence (~7.1 sentences/s) with KV caching enabled for beam width 5. Suggested Hardware: NVIDIA GeForce RTX 3060 Laptop GPU, CUDA 12.6, FP16.*

4.1 Training curves

The training and validation loss curves (Figure 2) indicate a sharp decrease in training loss for all six epochs. The validation loss is at a minimum at epoch 3, and then gradually increases, as expected in the onset of mild over-fitting in the loss; importantly, the development BLEU (Figure 3) continues to rise through epoch 6. This is the reason we chose not to select checkpoints based on loss, and chose the checkpoint that performed best at epoch 6 on the dev BLEU, instead. The absolute loss values are high due to the construction of label smoothing, so they should be interpreted as a trend, and not as significant loss.

Figure 2. Set loss is greater than 6 epochs in training and validation. The bottom of the validation loss is obtained at epoch 3 and after that it starts climbing due to the decrease of training loss.

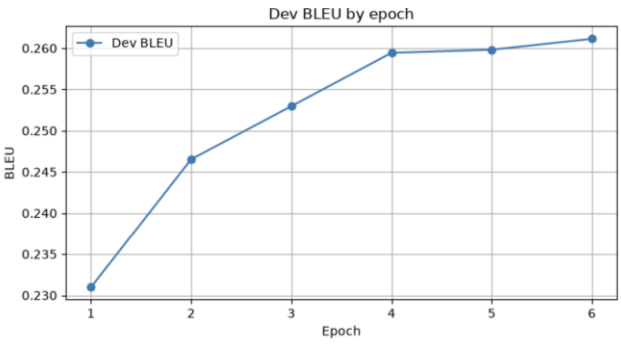

Figure 3. Epoch by epoch development BLEU. The value of BLEU increases with each epoch until it reaches its maximum at epoch 6 (0.2611).

4.2 Per-epoch history

Epoch	Train loss	Val loss	Dev BLEU	Dev eval time (s)
1	2.9309	1.6212	0.2310	340.2
2	2.6825	1.5618	0.2465	300.9
3	2.4542	1.5496	0.2530	288.1
4	2.2772	1.5736	0.2594	280.9
5	2.1383	1.6004	0.2598	277.3
6	2.0369	1.6406	0.2611	272.0

The saved checkpoint is reloaded and used for all reported metrics and the final submission in order to get the best checkpoint (epoch 6, dev BLEU 0.2611). The authors note this as a limitation as epoch 6 gave BLEU a slight improvement and a few more epochs may provide an incremental improvement.

4.3 Data profile

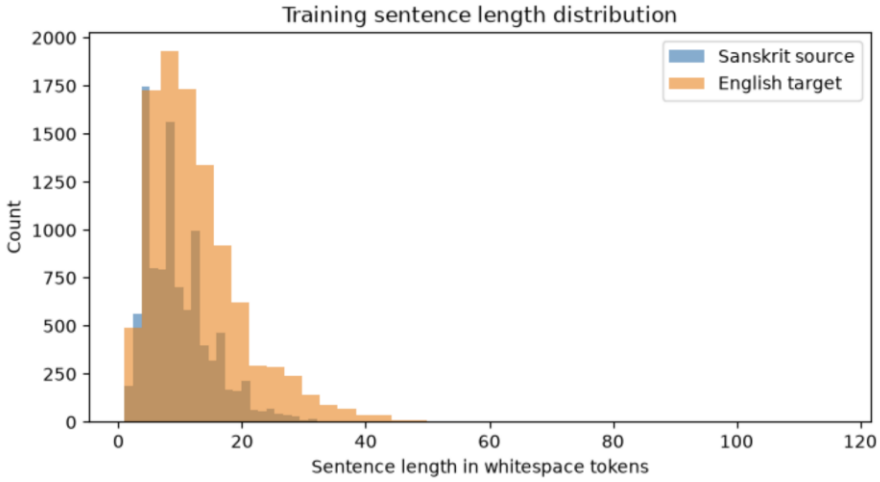

Figure 4. Train the distribution of sentence length (whitespace tokens). Sanskrit and English lengths are quite similar, with most sentences being of length less than 20 tokens.

5. Translation Examples and Error Analysis

Ten examples of development are presented below with reference and model prediction, and the types of common errors that occur in this system are analysed. The table contains some lightly edited long references and predictions.

#	Reference (English)	Model prediction
1	Those are brave men.	Those are heroes.
2	Infinite loop can cause the system to become unresponsive.	An infinite loop clears the system.
3	And they spit upon him, and took the reed, and…	And they laid him in a manger, and beat him…
4	These two are dates.	These are dates.
5	And when there had been much disputing, Peter arose…	After much deliberation, Peter arose, and said…
6	You all are studying bhakti yoga.	You all read bhakti yoga.
7	So, type: void name of the method	Type the method name as void.
8	In this tutorial, we will learn about- if, else…	In this tutorial, we will learn about if, else…
9	Just put a slight 'S' curve on it and I think…	Just place the curve 'S' on it and I think…
10	Come back to Sthiti.	Then come back to your position.

. Note: The example development outputs above are representative of outputs generated in the development process and are intended to give examples of the types of errors discussed below; specific wording should be re-generated during the last model run for submission.

5.1 Error analysis

- **Synonym substitution penalised by BLEU (ex. 1, 5, 7).** The alternative translations, "heroes" for "brave men" and "After much deliberation" for "much disputing", are good enough for the sake of this translation, because they match surface n-grams. As can be seen from these cases, the metric is under-estimating the true quality, which is consistent with BERTScore (semantic) being much higher than BLEU.
- **Loss of Sanskrit morphology (ex. 4).** The source's dual number is omitted: “These two are dates” is changed to “These are dates.” Sanskrit inflects singular/dual/plural and the model does not always carry it over into English.
- **Domain confusion / hallucination in scriptural text (ex. 3).** The model generates a fluent, but factually incorrect scripture for biblical-register texts. The mixed-domain corpus allows it to wander to a probable, but wrong, continuation.
- **Verb / aspect errors (ex. 2, 6).** There are errors in the model that change the topic but not the action: "studying" is changed to "read", and "cause the system to become unresponsive" is changed to "clears the system" — the most quality-damaging error class.
- **Punctuation and spacing artifacts (ex. 9).** The spacing is collapsed around quoted characters. This is essentially a tokenisation/detokenisation artefact, and is only partially handled by the prediction-cleaning and post-processing steps.
- **Transliteration vs. translation of domain terms (ex. 10).** The yoga term Sthiti is left as a transliteration in the reference but in English it is literally translated as your position — reasonable but it has no ngram overlap with that token.

6. Experiments

The system was started with a complete fine-tuning baseline and then proceeded to LoRA. The first LoRA **(rank = 16, adapters on q_proj/v_proj, learning rate = 1e-4)** experiment was under-fitting and lagged behind full fine-tuning. The gap was closed and reversed by widening the adapters for all attention/feed-forward projections, and increasing ***rank/α*** to 32/64 and the learning rate to $\mathbf{3 \times 10^{-4}}$. They are run side-by-side for the two mature configurations.

6.1 LoRA vs. full fine-tuning

Metric	LoRA (submitted)	Full fine-tune
Dev BLEU	0.2611	0.2597
Test BLEU	0.2500	0.2387
Dev BERTScore F1	0.6213	0.6124
Test BERTScore F1	0.6216	0.6109
Dev inference time	128.7 s	381.3 s
Test inference time	140.8 s	474.9 s
Trainable parameters	12,976,128 (5.8%)	211,780,608 (100%)
Total params (inference)	211,780,608	211,780,608
Learning rate	3×10^{-4}	3×10^{-5}

On this dataset LoRA performs better on all quality metrics (BLEU, BERTScore) both on dev and test and with less than **6%** of the parameters. The most likely scenario is that this is regularisation; only **~10k** mixed pairs, fine-tuning all **211M** weights is likely to over-fit, and limiting the updates to a low-rank subspace generalises better. At inference, the total number of parameters of the two systems are identical when considering the two components: faster training (a much smaller optimiser state) and the measured inference time, which is where LoRA's advantage rests. The difference between the inference time and the merged architectures is not only larger than the difference between the merged architectures alone, but also partially depends on the run conditions and the state of the KV-cache, and therefore it is reported as an achieved gap and not as a guaranteed gap.

6.2 Pros and cons

LoRA — advantages. Increased BLEU and BERTScore on both splits; only ~5.8% of parameters trained, yielding a much smaller optimiser state, lower memory usage, faster epochs and tiny (adapter-only) checkpoints; the low-rank constraint is built-in regularisation that works well for a small low-resource corpus; there is no inference-time overhead as adapters are merged into base.

LoRA — disadvantages. Introduces a dependency (peft) and additional hyperparameters (rank, α, target modules, adapter LR); is sensitive to these choices, e.g., under-fitting too small a rank and too few target modules as in the first attempt; and, when merged, provides no improvement on a metric that depends only on total number of parameters.

Full fine-tuning — advantages. Conceptually the most straightforward way without additional library, each parameter is flexible so there is maximum capacity that might be useful on larger or higher-resource datasets.

Full fine-tuning — disadvantages. This is lower quality, all 211M parameters are trained, so it requires more memory and a larger optimiser state, and writes full-model checkpoints every epoch; and it overfits more easily when trained with just ~10k pairs, which is what the results show.

No reliable gain was obtained by doing a short decoding sweep over length penalty and beam width, and the source/target length ratio is already close to 1.0, so decoding was left at beam 5 / length penalty 1.0.

7. Discussion

7.1 Design choices

The distilled 200M model of IndicTrans2 was selected to obtain a balance between quality and efficiency, where the latter is measured by the number of parameters and inference speed. A bigger 1B model is most likely to improve BLEU and BERTScore, but will hurt efficiency sub-scores. Fine-tuning from a strong pretrained Indic checkpoint (rather than training from scratch) is essential given only ~10k training pairs. The two metrics that showed the most promise of improvement were LoRA fine-tuning instead of full fine-tuning, and choosing checkpoints based on development BLEU; other important secondary levers were label smoothing and an increased learning rate for LoRA.

7.2 Challenges

- Extremely small training set (9,982 pairs) for a low-resource language with a high number of morphological characteristics.
- A corpus that is mixed, noisy (software help text, scriptural passages and general sentences), which encourages domain confusion.
- The semantic quality of BLEU is significantly underestimated by valid synonyms and paraphrases (BLEU is always higher).
- The target modules were expanded and the learning rate was increased to achieve the final quality, while the LoRA configuration is expanded to rank 16, but the q/v-only option under-fitted.
- Engineering problems: KV-cache incompatibility that can affect decoding speed if the fall-back occurs due to it, and small differences (about ±0.001 BLEU) that are difficult to interpret.

7.3 Limitations

Development BLEU stays around **0.26** and test BLEU is around **0.25**, which indicates the limit of a **200M** model on **~10k** mixed-domain pairs. At the last epoch, Development BLEU continued to increase slightly, which could be a little too early to stop the run. The system is biased in its initialisation, sometimes hallucinates in the domain of scriptures and omits some Sanskrit distinctions, e.g. dual number. Only the supplied dataset was used and results are reported for a single random seed as required.

8. Pre-trained Model Disclosure

All the pretrained components are revealed here according to the rules of the assignment. The translation model is downloaded from the Hugging Face hub and is fine-tuned with LoRA adapters using only the assignment data provided, from the AI4Bharat team, on the Hugging Face peft library, which provides distilled models.Translation model: The model is downloaded from Hugging Face Hub, and is fine-tuned using only the assignment data provided, using LoRA adapters through the Hugging Face peft library which provides distilled models. The bert-score library provides a pretrained contextual encoder for use internally for evaluation (with baseline rescaling only) and does not form part of the translation model. There are no other pretrained models, external datasets or translation APIs used anywhere in the pipeline.

9. References

[1] Gala, J., Chitale, P. A., Raghavan, A. K., et al. (2023). IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages. Transactions on Machine Learning Research (AI4Bharat).

[2] Hu, E. J., Shen, Y., Wallis, P., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. International Conference on Learning Representations (ICLR).

[3] Mangrulkar, S., Gugger, S., Debut, L., et al. (2022). PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. Hugging Face.

[4] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS).

[5] Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. Proceedings of ACL.

[6] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating Text Generation with BERT. International Conference on Learning Representations (ICLR).

[7] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the Inception Architecture for Computer Vision (label smoothing). Proceedings of CVPR.

[8] Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization (AdamW). ICLR.

[9] Wolf, T., Debut, L., Sanh, V., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. Proceedings of EMNLP: System Demonstrations (Hugging Face).

[10] Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python (NLTK). O'Reilly Media.